

計算機網路期中專案說明

網站離線下載工具

系級：資工三甲

學號：B1222022

姓名:洪碩廷

課程名稱：計算機網路

授課教師：李春良 教授

題目摘要：

1. 要做一個「簡易版的網站下載器」
2. 使用者輸入網址，就能把整個網站下載下來（包含 HTML、圖片、PDF）
3. 支援離線瀏覽：連結都會被改成本地路徑
4. 不可以用現成函式庫（要自己寫 socket 傳送 HTTP 請求）

主要程式碼架構說明：

檔案	負責內容說明
main.cpp	程式起點、處理參數、呼叫爬蟲
config.h	處理使用者輸入的選項（-u -d -e -c 等）
crawler.cpp	整個網站爬蟲流程、BFS 遞迴、重寫 HTML
http_client.cpp	負責 socket 連線與傳送 HTTP 請求、接收檔案
url.cpp	分析網址、合成完整 URL、轉換成本地路徑
crawler.h, http_client.h, url.h	對應的函式宣告與資料結構

程式碼核心架構說明與解釋：

程式進入點（main.cpp）

程式碼摘要: main.cpp 是使用者操作入口，所有設定都在這裡決定後才交給爬蟲執行。

```
if (!has_args || cfg.url_str.empty()) {  
    // 互動模式  
    interactive_input(cfg);  
}  
  
if (cfg.output_dir.empty()) {  
    cfg.output_dir = "download";  
}  
  
print_config(cfg);  
  
std::cout << "起始 URL: " << cfg.url_str << "\n";  
  
crawl_site(cfg);  
  
std::cout << "全部下載流程結束。\\n";  
return 0;  
}
```

1. 程式從這裡開始執行 (int main)。
2. 呼叫 parse_args() 檢查是否用命令列模式執行。
3. 若沒帶參數，則改用互動式模式 (interactive_input())。
4. 設定下載輸出資料夾（預設 download）。
5. 印出設定內容後，呼叫 crawl_site(cfg) 啟動爬蟲流程。

設定與命令列解析(config.h)

程式碼摘要：config.h 負責「收集與記錄所有設定」，是程式運行前的控制中樞。

```
20 inline bool parse_args(int argc, char* argv[], Config& cfg) {
21     if (argc < 2) {
22         // 沒帶參數 → 交給 main() 做互動模式
23         return false;
24     }
25
26     for (int i = 1; i < argc; ++i) {
27         std::string arg = argv[i];
28         if (arg == "-u" && i + 1 < argc) {
29             cfg.url_str = argv[++i];
30         } else if ((arg == "-o" || arg == "-o") && i + 1 < argc) {
31             cfg.output_dir = argv[++i];
32         } else if (arg == "-d" && i + 1 < argc) {
33             cfg.depth = std::stoi(argv[++i]);
34         } else if (arg == "-e" && i + 1 < argc) {
35             int v = std::stoi(argv[++i]);
36             cfg.allow_external = (v != 0);
37         } else if (arg == "-c" && i + 1 < argc) {
38             int v = std::stoi(argv[++i]);
39             cfg.resume = (v != 0);
40         } else {
41             std::cerr << "未知參數: " << arg << "\n";
42         }
43     }
44
45     return !cfg.url_str.empty();
46 }
47
```

1. 定義 Config 結構，儲存使用者設定（URL、深度、是否允許外站、續傳模式）。
2. print_config()：顯示目前設定狀態。

網址解析與重組(url.cpp):

程式碼摘要：url.cpp 是整個爬蟲正確定位連結的基礎。沒有這層解析，爬蟲會無法處理相對路徑。

```
if (url.compare(0, prefix.size(), prefix) != 0) {  
    return false;  
}  
out.scheme = "http";  
  
std::size_t pos = prefix.size();  
std::size_t slash_pos = url.find('/', pos);  
std::string host_port;  
if (slash_pos == std::string::npos) {  
    host_port = url.substr(pos);  
    out.path = "/";  
} else {  
    host_port = url.substr(pos, slash_pos - pos);  
    out.path = url.substr(slash_pos);  
}
```

1. parse_url(): 將完整網址分解為 host、path、port。
2. join_url(): 將 base URL 與相對路徑合併為完整網址。

```
std::size_t pos = prefix.size();  
std::size_t slash_pos = url.find('/', pos);  
std::string host_port;  
if (slash_pos == std::string::npos) {  
    host_port = url.substr(pos);  
    out.path = "/";  
} else {  
    host_port = url.substr(pos, slash_pos - pos);  
    out.path = url.substr(slash_pos);  
}  
  
std::size_t colon_pos = host_port.find(':');  
if (colon_pos == std::string::npos) {  
    out.host = host_port;  
    out.port = 80;  
} else {  
    out.host = host_port.substr(0, colon_pos);  
    std::string port_str = host_port.substr(colon_pos + 1);  
    try {  
        out.port = std::stoi(port_str);  
    } catch (...) {  
        // 忽略轉換錯誤，使用預設 port  
        out.port = 80;  
    }  
}  
  
if (out.path.empty())  
    out.path = "/";  
  
return true;
```

HTTP 資料下載(http_client.cpp):

程式碼摘要：這是最底層、實際與網站連線並取得資料的模組

```
// 2. 建立 socket & connect
int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
if (sock < 0) {
    std::perror("socket");
    freeaddrinfo(res);
    return false;
}

if (connect(sock, res->ai_addr, res->ai_addrlen) < 0) {
    std::perror("connect");
    close(sock);
    freeaddrinfo(res);
    return false;
}

freeaddrinfo(res);

// 3. 組 GET
std::string path = url.path.empty() ? "/" : url.path;
std::string req =
    "GET " + path + " HTTP/1.1\r\n"
    "Host: " + url.host + "\r\n"
    "Connection: close\r\n"
    "\r\n";

const char* p = req.c_str();
size_t left = req.size();
while (left > 0) {
    ssize_t n = send(sock, p, left, 0);
    if (n <= 0) {
        std::perror("send");
        close(sock);
        return false;
    }
    p += n;
    left -= n;
}
```

```
while (true) {
    ssize_t n = recv(sock, buf, BUF_SIZE, 0);
    if (n < 0) {
        std::perror("recv");
        std::fclose(fp);
        close(sock);
        return false;
    }
    if (n == 0) break;

    if (!header_done) {
        header_buf.append(buf, n);
        std::size_t pos = header_buf.find("\r\n\r\n");
        if (pos != std::string::npos) {
            // 解析 Content-Length
            std::string header_text = header_buf.substr(0, pos);
            std::size_t cl_pos = header_text.find("Content-Length:");
            if (cl_pos != std::string::npos) {
                cl_pos += 15;
                while (cl_pos < header_text.size() && (header_text[cl_pos] == ' ' || header_text[cl_pos] == '\t'))
                    ++cl_pos;
                std::size_t end = cl_pos;
                while (end < header_text.size() && header_text[end] >= '0' && header_text[end] <= '9')
                    ++end;
                std::string num = header_text.substr(cl_pos, end - cl_pos);
                if (!num.empty()) {
                    content_length = static_cast<std::size_t>(std::stoll(num));
                    has_content_length = true;
                }
            }
        }
    }
}
```

1. 建立 socket → 用 `getaddrinfo()` 找主機 IP，接著用 `connect()` 連線。
2. 送出 HTTP GET 請求 → 例如：
3. 接收回應資料 (`recv`)
 - 先解析 HTTP 標頭，找 Content-Length。
 - 接著將 HTML / 圖片 / CSS 內容寫入本地檔案 (`fwrite`)。
4. 顯示即時進度與下載速度。

爬蟲主流程（crawler.cpp）：

程式碼摘要：crawler.cpp 是整個系統的「腦袋」，控制何時抓取、抓誰、抓到哪一層，它整合了所有模組（config、url、http_client）。

```
//BFS
while (!q.empty()) {
    Node cur = q.front();
    q.pop();

    Url u;
    if (!parse_url(cur.url_str, u)) continue;

    std::string local_path = url_to_local_path(u, cfg.output_dir);

    bool is_html = (local_path.size() >= 4 &&
                    (local_path.substr(local_path.size() - 4) == ".htm" ||
                     local_path.substr(local_path.size() - 5) == ".html"));
    bool is_css = (local_path.size() >= 4 && local_path.substr(local_path.size() - 4) == ".css");

    bool should_download = true;

    if (cfg.resume && file_exists(local_path)) {
        // -c 1: 繼續爬取，若該檔案已存在
        std::cout << "[繼續已存在] " << local_path << "\n";
        should_download = false;
    }

    if (should_download) {
        std::cout << "\n[取回] depth=" << cur.depth << " " << cur.url_str
                  << " -> " << local_path << "\n";

        // http_download_to_file 會自動建立父目錄
        bool ok = http_download_to_file(u, local_path, &global_stats);
        if (!ok) {
            std::cerr << "[下載失敗] " << cur.url_str << "\n";
            continue;
        }
    }
}
```

```
// 若深度已達上限（僅針對 HTML 頁面），則不再解析
if (cur.depth >= cfg.depth && is_html) continue;

// 僅對 HTML 和 CSS 檔案進行解析/重寫
if (!is_html && !is_css) continue;

// 讀檔、準備解析/重寫（使用 std::ios::binary 確保讀取二進制資料完整）
std::ifstream fin(local_path.c_str(), std::ios::binary);
if (!fin) {
    std::cerr << "[錯誤] 無法讀取檔案以進行解析/重寫: " << local_path << "\n";
    continue;
}

std::string content((std::istreambuf_iterator<char>(fin),
                    std::istreambuf_iterator<char>()));
fin.close();

// 核心：連結重寫（實作離線瀏覽要求）
std::string processed_content = rewrite_content(content, u, local_path, cfg.output_dir);

// 重新寫回檔案（使用 std::ios::binary 確保寫入二進制資料完整）
std::ofstream fout(local_path.c_str(), std::ios::trunc | std::ios::binary);
if (fout) {
    fout.write(processed_content.data(), processed_content.size());
    fout.close();
} else {
    std::cerr << "[警告] 無法重新寫入檔案以重寫連結: " << local_path << "\n";
}
```

1. 準備階段，刪除舊資料夾並初始化 BFS 佇列與 visited 集合。
2. 核心流程。(BFS 遞迴爬取)

每次取出一個連結下載。

若是 HTML/CSS → 呼叫 extract_links() 擷取新連結。

若深度未超過限制 → 加入 queue 等待下一輪。

3. 連結重寫與離線修正：

讀取 HTML 內容後，用 rewrite_content() 改寫內部連結。

讓離線檔案仍能互相連通。

```
std::cout << "\n=== 下載統計資訊 ===\n";
std::cout << " 已下載檔案數量 : " << global_stats.total_files << "\n";
std::cout << " 已下載總容量 : " << global_stats.total_bytes << " bytes\n";
std::cout << " 已下載時間 : " << elapsed << " 秒\n";
std::cout << "=====\n";
```

4. 統計輸出

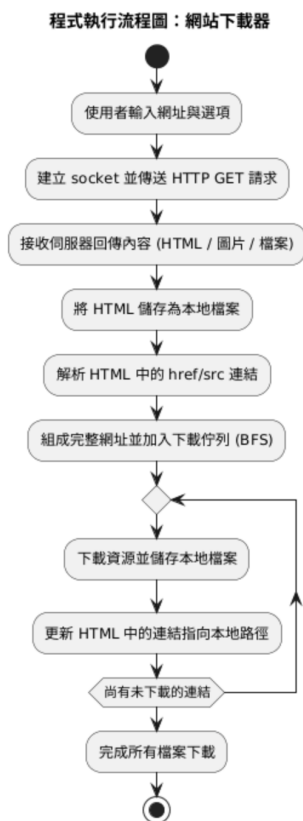
宣告主要介面(crawler.h):

程式碼摘要：讓各個模組之間能安全地呼叫爬蟲主功能，避免重複

定義或交叉依賴

```
1 // crawler.h
2 #ifndef CRAWLER_H
3 #define CRAWLER_H
4
5 #include "config.h"
6 #include "http_client.h"
7 #include "url.h"
8 #include <string>
9
10 // 新增：宣告一個用於計算 URL 存到本機後路徑的輔助函式
11 std::string url_to_local_path(const Url& url, const std::string& output_dir);
12
13 void crawl_site(const Config& cfg);
14
15 #endif // CRAWLER_H
```

程式執行流程圖：



操作說明：

本次檔案夾專案皆以（Project_root）進行命名

使用者操作如以下

命令模式終端機輸入：

```
./downloader -u http://hsccl.fr.to/index.htm -O download -d 2 -e 0 -c 1
```

參數	意義	範例
-u	指定起始 URL	http://hsccl.fr.to/index.htm
-O	指定輸出目錄	download
-d	指定遞迴深度 (0 表示只抓首頁)	2
-e	是否允許外部網站連結 (1=是, 0=否)	0
-c	續傳設定 (1=略過已下載, 0=重新下載)	1

執行後，程式會自動：

1. 下載指定網站與其所有相關檔案。
2. 解析 HTML 中的所有 <a>、、<link> 等連結。
3. 自動修改連結為本機相對路徑，確保離線可瀏覽。
4. 顯示下載進度與統計資訊（檔案數量、總容量、花費時間）。

互動模式終端機輸入:

./downloader

```
=== Downloader 互動模式 ===
請輸入 URL: http://hsccl.fr.to/index.htm
請輸入輸出目錄 (例如 download): download
請輸入遞迴深度 depth (0 = 不遞迴): 2
是否允許外站? (0 = 否, 1 = 是): 0
續傳模式? (0 = 重來並刪除舊目錄, 1 = 繼續下載): 1
[命令列模式] 下載設定 =====
URL          : http://hsccl.fr.to/index.htm
輸出目錄     : download
深度 (depth) : 2
允許外站     : 否
續傳模式 (-c) : 繼續下載
=====
起始 URL: http://hsccl.fr.to/index.htm
[跳過已存在] download/index.htm
[跳過已存在] download/images/animals/cats/image.jpg
[跳過已存在] download/image.jpg
[跳過已存在] download/images/image.jpg

=== 下載統計資訊 ===
已下載檔案數量 : 0
已下載總容量   : 0 bytes
已下載時間     : 0.00132508 秒
=====
```

輸出結果：

1. 所有下載內容將儲存在指定的輸出資料夾（例如 download/）。
2. 每個子頁面、圖片、CSS、PDF 皆會自動依路徑層級儲存。
3. 完成後在終端機輸出統計資訊

參考資料：

李春良 (2025)。〈Computer Networks 課程講義與專題說明〉。長庚大學 資訊工程學系。

李春良 (2025)。〈Computer Networks 計算機網路 Chapter 講義〉。長庚大學 資訊工程學系。

Lin, Y.-D., Hwang, R.-H., & Baker, F. (2012)。 *Computer Networks: An Open Source Approach*。 McGraw Hill。 ISBN：978-00-07-131587-6。

OpenAI (2025)。〈ChatGPT (GPT-5 模型) 輔助除錯〉。取自 <https://chat.openai.com> (最後存取日期：2025 年 11 月 27 日)。